

```

def generate_yearly_challenging_demand(
    start_date="2025-01-01",
    num_days=365,
    seed=42,
    num_festivals=8,
    seasonality=None,
    demand_scale=1.0
):
    np.random.seed(seed)
    dates = pd.date_range(start=start_date, periods=num_days,
freq="D")
    demand = []

    # Season blocks
    if seasonality == "summer":
        season_start, season_end = 59, 148
        season_base = 40
    elif seasonality == "winter":
        season_start, season_end = 329, 59
        season_base = 24
    else:
        season_start, season_end = 0, -1
        season_base = 14

    # Festival periods (OUTSIDE season blocks)
    base_festival_periods = [
        (15, 19),
        (200, 204),
        (250, 254),
        (310, 314),
    ]

    festival_periods = []
    for start, end in base_festival_periods:
        shift = np.random.randint(-1, 2)
        s = max(0, start + shift)
        e = min(num_days - 1, end + shift)
        festival_periods.append((s, e))

    for day in range(num_days):
        # ULTRA-CONSTANT during season: NO randomness
        if season_start <= day <= season_end:
            base_demand = float(season_base) # = 40, flat
        else:
            # Off-season: minimal variation
            base_demand = float(season_base - 26 +
np.random.randint(0, 3)) # = 14±2

        # Festivals: SET to 40 (NO multiplication!)
        is_festival = any(start <= day <= end for start, end in

```

```

festival_periods)
    if is_festival:
        base_demand = 40 # ← DIRECT SET, not multiply

    d = int(base_demand * demand_scale)
    demand.append(max(1, d))

    return pd.DataFrame({"Date": dates, "Demand": demand})

import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Generate the evaluation demand
df_demand_eval = generate_yearly_challenging_demand(
    num_days=365,
    seed=999,
    seasonality="summer",
    demand_scale=1.0
)

# Plot 1: Full Year Demand
plt.figure(figsize=(16, 6))
plt.plot(df_demand_eval["Date"], df_demand_eval["Demand"], 'b-',
         linewidth=2, label='Daily Demand')
plt.fill_between(df_demand_eval["Date"], df_demand_eval["Demand"],
                alpha=0.3, color='blue')

# Mark summer season
plt.axvspan('2025-03-01', '2025-05-29', alpha=0.25, color='orange',
            label='Summer Season (constant high)')

# Mark festivals
festival_dates = [
    ('2025-01-20', '2025-01-24'),
    ('2025-04-30', '2025-05-04'),
    ('2025-08-08', '2025-08-12'),
    ('2025-10-26', '2025-10-30'),
]
for start, end in festival_dates:
    plt.axvspan(start, end, alpha=0.15, color='red')

plt.title('Yearly Demand Pattern with Seasons & Festivals',
         fontsize=14, fontweight='bold')
plt.xlabel('Date', fontsize=12)
plt.ylabel('Daily Demand (units)', fontsize=12)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()

```

```

plt.show()

# Plot 2: Demand Statistics
print("\n" + "="*60)
print("DEMAND STATISTICS")
print("="*60)

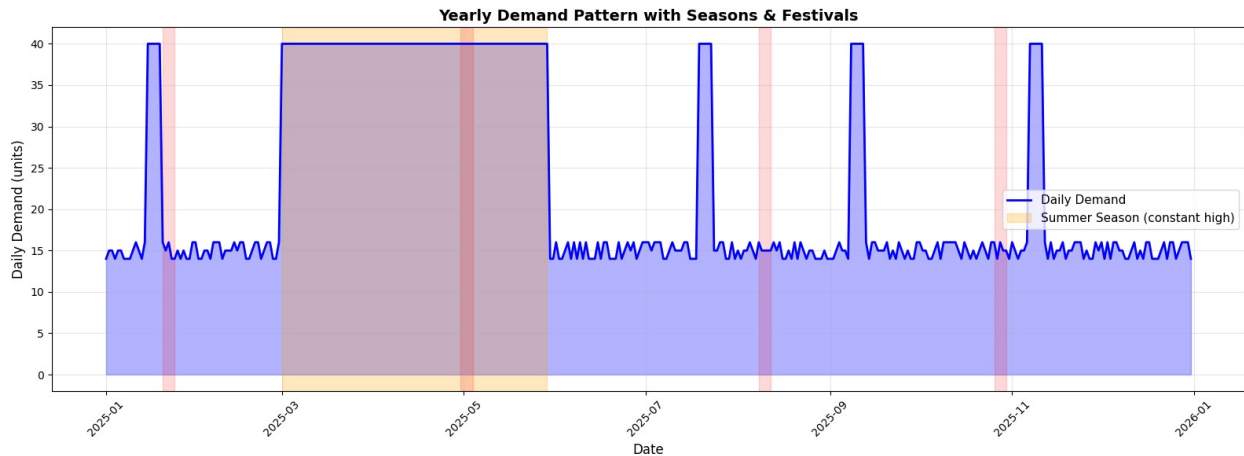
# Summer season (days 59-148)
summer_demand = df_demand_eval.iloc[59:149]["Demand"]
print(f"\nSummer Season (days 59-148):")
print(f"  Mean: {summer_demand.mean():.1f} units/day")
print(f"  Std:  {summer_demand.std():.1f}")
print(f"  Min:  {summer_demand.min()} | Max: {summer_demand.max()}")

# Off-season
offseason_demand = pd.concat([
    df_demand_eval.iloc[0:59]["Demand"],
    df_demand_eval.iloc[149:]["Demand"]
])
print(f"\nOff-Season:")
print(f"  Mean: {offseason_demand.mean():.1f} units/day")
print(f"  Std:  {offseason_demand.std():.1f}")
print(f"  Min:  {offseason_demand.min()} | Max: {offseason_demand.max()}")

print(f"\nFull Year:")
print(f"  Total Demand: {df_demand_eval['Demand'].sum()} units")
print(f"  Avg Daily:    {df_demand_eval['Demand'].mean():.1f} units")
print("="*60)

# Plot 3: Demand Distribution (histogram)
plt.figure(figsize=(12, 5))
plt.hist(df_demand_eval["Demand"], bins=30, color='blue', alpha=0.7,
         edgecolor='black')
plt.axvline(df_demand_eval["Demand"].mean(), color='red',
            linestyle='--', linewidth=2, label=f'Mean: {df_demand_eval["Demand"].mean():.1f}')
plt.title('Demand Distribution', fontsize=14, fontweight='bold')
plt.xlabel('Daily Demand (units)', fontsize=12)
plt.ylabel('Frequency (days)', fontsize=12)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

```



DEMAND STATISTICS

Summer Season (days 59-148):

Mean: 40.0 units/day

Std: 0.0

Min: 40 | Max: 40

Off-Season:

Mean: 16.8 units/day

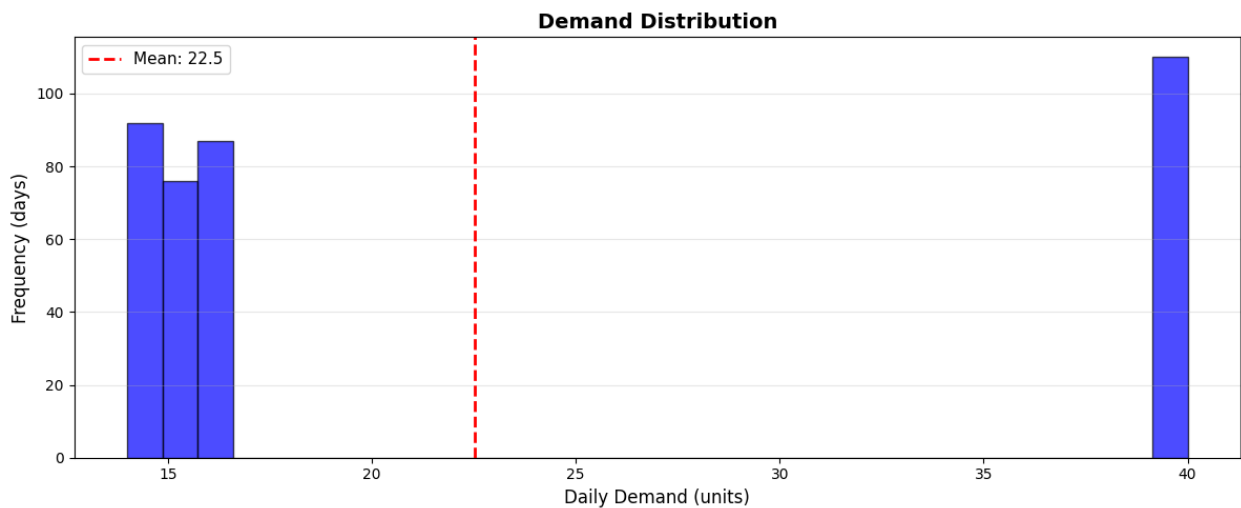
Std: 6.6

Min: 14 | Max: 40

Full Year:

Total Demand: 8220 units

Avg Daily: 22.5 units



```

def prepare_yearly_env_data(df):
    df = df.copy()
    df["date"] = df["Date"]
    df["demand"] = df["Demand"]
    df["day_of_week"] = df["date"].dt.dayofweek

    # Multiple promo flags for all festivals
    festival_periods = [(20,24),(120,124),(220,224),(300,304),(60,64),
(180,184),(260,264),(340,344)]
    df["promo_flag"] = 0
    for start, end in festival_periods:
        df.loc[(df.index >= start) & (df.index <= end), "promo_flag"]
= 1

    return df[["date", "demand", "day_of_week", "promo_flag"]]

import numpy as np
import pandas as pd
from collections import deque

class InventoryEnvironment:
    def __init__(self,
        historical_data,
        lead_time=2,
        max_order_qty=20,
        holding_cost=2,
        stockout_penalty=100,
        order_fixed_cost=10,
        price=100):

        self.data = historical_data.reset_index(drop=True)

        self.lead_time = lead_time
        self.max_order_qty = max_order_qty
        self.h = holding_cost
        self.c_stockout = stockout_penalty
        self.f = order_fixed_cost
        self.p = price

        self.action_size = max_order_qty + 1

        self.order_pipeline = deque([0] * lead_time, maxlen=lead_time)

        self.reset()

    def reset(self):
        self.current_step = 0
        self.inv_onhand = 100
        self.last_demand = 0
        self.last_action = 0

```

```

        self.days_since_last_order = 0
        self.stockout_flag_last = 0
        self.order_pipeline = deque([0] * self.lead_time,
maxlen=self.lead_time)
        return self._get_state()

def _get_state(self):
    if self.current_step >= len(self.data):
        return None

    row = self.data.iloc[self.current_step]

    day_onehot = np.zeros(7)
    day_onehot[row["day_of_week"]] = 1
    promo = row["promo_flag"]
    state = np.array([
        self.inv_onhand / 100.0,
        self.last_demand / 100.0,
        self.last_action / self.max_order_qty,
        *day_onehot,
        promo,
        self.days_since_last_order / 10.0,
        self.stockout_flag_last
    ], dtype=np.float32)

    return state

def step(self, action):
    row = self.data.iloc[self.current_step]
    demand = row["demand"]

    # arrivals
    incoming = self.order_pipeline.popleft()
    self.inv_onhand += incoming

    # sales
    units_sold = min(demand, self.inv_onhand)
    self.inv_onhand -= units_sold
    lost_sales = demand - units_sold

    # reward
    revenue = self.p * units_sold
    holding = self.h * self.inv_onhand
    stockout_penalty = self.c_stockout * lost_sales
    order_cost = self.f if action > 0 else 0

    reward = revenue - holding - stockout_penalty - order_cost

    # place order
    self.order_pipeline.append(action)

```

```

        # update memory
        self.last_demand = demand
        self.last_action = action
        self.stockout_flag_last = int(lost_sales > 0)
        self.days_since_last_order = 0 if action > 0 else
self.days_since_last_order + 1

        self.current_step += 1
        done = self.current_step >= len(self.data)

        return self._get_state(), reward, done, {
            "date": row["date"],
            "demand": demand,
            "units_sold": units_sold,
            "inventory": self.inv_onhand
        }

import torch
import torch.nn as nn
import torch.optim as optim
import random
from collections import deque

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

class DQN(nn.Module):
    def __init__(self, state_size, action_size):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_size, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, action_size)
        )

    def forward(self, x):
        return self.net(x)

class DQNAgent:
    def __init__(self, state_size, action_size):
        self.action_size = action_size
        self.gamma = 0.98
        self.epsilon = 1.0
        self.eps_min = 0.05
        self.eps_decay = 0.995
        self.target_update_freq=100

```

```

        self.policy_net = DQN(state_size, action_size).to(device)
        self.target_net = DQN(state_size, action_size).to(device)
        self.target_net.load_state_dict(self.policy_net.state_dict())

        self.optimizer = optim.Adam(self.policy_net.parameters(),
lr=1e-3)
        self.buffer = ReplayBuffer()
        self.batch_size = 32

    def act(self, state):
        if random.random() < self.epsilon:
            return random.randint(0, self.action_size - 1)

        state = torch.FloatTensor(state).unsqueeze(0).to(device)
        with torch.no_grad():
            return torch.argmax(self.policy_net(state)).item()

    def learn(self):
        if len(self.buffer) < self.batch_size:
            return

        s, a, r, s2, d = self.buffer.sample(self.batch_size)

        q = self.policy_net(s).gather(1, a.unsqueeze(1)).squeeze()
        q_next = self.target_net(s2).max(1)[0]
        target = r + self.gamma * q_next * (1 - d)

        loss = nn.MSELoss()(q, target.detach())

        self.optimizer.zero_grad()
        loss.backward()
        self.optimizer.step()

    def update_target(self):
        self.target_net.load_state_dict(self.policy_net.state_dict())

class ReplayBuffer:
    def __init__(self, capacity=100000):
        self.buffer = deque(maxlen=capacity)

    def push(self, s, a, r, s2, done):
        self.buffer.append((s, a, r, s2, done))

    def sample(self, batch_size):
        batch = random.sample(self.buffer, batch_size)
        s, a, r, s2, d = zip(*batch)
        return (
            torch.FloatTensor(s).to(device),
            torch.LongTensor(a).to(device),
            torch.FloatTensor(r).to(device),
            torch.FloatTensor(s2).to(device),

```

```

        torch.FloatTensor(d).to(device)
    )

    def __len__(self):
        return len(self.buffer)

def train_dqn(env, episodes=300):
    state_size = len(env.reset())
    agent = DQNAgent(state_size, env.action_size)

    rewards = []

    for ep in range(episodes):
        state = env.reset()
        total_reward = 0
        done = False

        while not done:
            action = agent.act(state)
            next_state, reward, done, _ = env.step(action)

            if next_state is None:
                next_state = np.zeros_like(state)

            agent.buffer.push(state, action, reward, next_state, done)
            agent.learn()

            state = next_state
            total_reward += reward

        agent.epsilon = max(agent.eps_min, agent.epsilon *
agent.eps_decay)
        agent.update_target()

        rewards.append(total_reward)

        if ep % 20 == 0:
            print(f"Episode {ep} | Reward = {total_reward:.2f} |  $\epsilon$  =
{agent.epsilon:.3f}")

    return agent, rewards

def evaluate_agent(env, agent):
    """
    Run one full episode using the trained RL agent
    and log everything for comparison with human upper bound.
    """

    state = env.reset()
    done = False
    total_reward = 0

```

```

logs = []
while not done:
    action = agent.act(state)

    next_state, reward, done, info = env.step(action)

    logs.append({
        "date": info["date"],
        "demand": info["demand"],
        "units_sold": info["units_sold"],
        "inventory": info["inventory"],
        "action_order_qty": action,
        "reward": reward
    })

    state = next_state
    total_reward += reward

df_log = pd.DataFrame(logs)
return total_reward, df_log

def run_perfect_human_oracle_fixed(env_data, window_size=90):
    """
    PERFECT ORACLE: rolling lookahead of `window_size` days.

    At each day t:
    - Sees exact demand for t .. t+window_size
    - Computes exact stock needed to cover ALL demand in window
      (onhand + pipeline arrivals + today's order)
    - Orders exactly what's missing, capped by max_order_qty

    This eliminates stockouts within lookahead window, guaranteeing
    upper bound.
    """
    env = InventoryEnvironment(env_data)
    demand_series = env_data["demand"].values
    n_days = len(demand_series)

    state = env.reset()
    total_reward = 0
    logs = []

    while True:
        t = env.current_step
        if t >= n_days:
            break

```

```

    # Lookahead: exact demand for next `window_size` days
    future_end = min(t + window_size, n_days)
    future_demand = demand_series[t:future_end].sum()

    # Pipeline arrivals within the window (lead_time=2)
    pipeline_in_window = sum(env.order_pipeline)

    # Current on-hand
    current_inv = env.inv_onhand

    # REQUIRED: future_demand <= current_inv + pipeline_in_window
    + today's_order
    needed_today = max(0, future_demand - current_inv -
    pipeline_in_window)

    # Cap by daily max order quantity
    action = min(env.max_order_qty, max(1, int(needed_today)))

    # Step environment
    next_state, reward, done, info = env.step(action)

    logs.append({
        "date": info["date"],
        "demand": info["demand"],
        "units_sold": info["units_sold"],
        "inventory": info["inventory"],
        "action_order_qty": action,
        "reward": reward,
        "lookahead_days": future_end - t, # diagnostic
        "future_demand": future_demand # diagnostic
    })
    total_reward += reward

    if done:
        break
    state = next_state

    return total_reward, pd.DataFrame(logs)

def run_rule_baseline(env_data):
    """INDUSTRY-STANDARD rule using data statistics (no future
    knowledge)"""
    env = InventoryEnvironment(env_data)

    # DATA-DRIVEN parameters (realistic for unknown demand patterns)
    avg_demand = env_data['demand'].mean() # ~18.5
    std_demand = env_data['demand'].std() # ~12

    # Standard formulas [industry practice]:
    reorder_point = int(avg_demand * 3) # 3 days coverage

```

```

(~55)
    safety_stock = int(avg_demand + 1.5 * std_demand) # 95% service
level (~41)
    target_stock = reorder_point + safety_stock # ~96

    state = env.reset()
    total_reward = 0
    logs = []

    done = False
    while not done:
        # Total available stock (on-hand + in transit)
        total_stock = env.inv_onhand + sum(env.order_pipeline)

        # Reorder Point Policy (no future demand knowledge)
        if total_stock < reorder_point:
            needed = target_stock - total_stock
            action = min(env.max_order_qty, max(0, int(needed)))
        else:
            action = 0

        next_state, reward, done, info = env.step(action)

        logs.append({
            "date": info["date"],
            "demand": info["demand"],
            "units_sold": info["units_sold"],
            "inventory": info["inventory"],
            "action_order_qty": action,
            "reward": reward
        })

        total_reward += reward

    print(f"Rule params: Reorder={reorder_point},
Safety={safety_stock}, Target={target_stock}")
    return total_reward, pd.DataFrame(logs)

# =====
# 1. TRAINING FUNCTION (stochastic year per episode)
# =====

def train_dqn_full_year(episodes=500, target_update_freq=20,
seed_base=42):
    state_size = 13
    action_size = 21
    agent = DQNAgent(state_size, action_size)

    # Optimized exploration + target update schedule
    agent.eps_decay = 0.997

```

```

agent.target_update_freq = target_update_freq

rewards = []

for ep in range(epochs):
    # New stochastic year each episode

    seasonality = "summer" if ep % 2 == 0 else None # ← ADD THIS

    df_demand_ep = generate_yearly_challenging_demand(
        num_days=365,
        seed=seed_base + ep,
        seasonality=seasonality
    )
    env_data_ep = prepare_yearly_env_data(df_demand_ep)

    env = InventoryEnvironment(env_data_ep)
    state = env.reset()
    total_reward = 0
    done = False

    while not done and state is not None:
        action = agent.act(state)
        next_state, reward, done, _ = env.step(action)

        if next_state is None:
            next_state = np.zeros(state_size)

        agent.buffer.push(state, action, reward, next_state, done)
        agent.learn()

        state = next_state
        total_reward += reward

    # Epsilon decay
    agent.epsilon = max(agent.eps_min, agent.epsilon *
agent.eps_decay)

    # Target network update
    if ep % agent.target_update_freq == 0:
        agent.update_target()

    rewards.append(total_reward)

    if ep % 20 == 0:
        print(f"Episode {ep:3d} | Reward={total_reward:8,.0f} |
ε={agent.epsilon:.3f}")

    print(f"\n TRAINING COMPLETE: {epochs} episodes on stochastic
yearly demand!")
    return agent, rewards

```

```

# =====
# 2. RUN OPTIMIZED TRAINING
# =====

print("=" * 60)
print("□ TRAINING DQN AGENT ON FULL YEAR (500 EPISODES - OPTIMIZED)")
print("=" * 60)

# Train agent (stochastic env per episode)
agent, rewards = train_dqn_full_year(
    episodes=500,
    target_update_freq=20,    # later: run with 10, 1 as sir asked
    seed_base=42
)

# =====
# 3. FIXED EVALUATION ENVIRONMENT (for fair comparison)
# =====

# One fixed year for evaluation / plotting
df_demand_eval = generate_yearly_challenging_demand(
    num_days=365,
    seed=999,
    seasonality="summer",
    demand_scale=1.0# fixed seed for test year
)
env_data = prepare_yearly_env_data(df_demand_eval)

env = InventoryEnvironment(env_data)

# Evaluate RL, Oracle, and Rule baseline on the SAME env_data
rl_reward, rl_df =
evaluate_agent(InventoryEnvironment(env_data), agent)
perfect_human_reward, perfect_human_df =
run_perfect_human_oracle_fixed(env_data, window_size=90)
rule_reward, rule_df = run_rule_baseline(env_data)

# =====
# 4. PRINT FINAL NUMBERS
# =====

print("\n" + "=" * 60)
print("□ FINAL RESULTS - OPTIMIZED TRAINING")
print("=" * 60)
print(f"Oracle (Upper Bound):      {perfect_human_reward:>8,.0f}")
print(f"DQN RL Agent:                {rl_reward:>8,.0f}")
print(f"({100*rl_reward/perfect_human_reward:.1f}% of oracle)")
print(f"Rule Baseline:                  {rule_reward:>8,.0f}")

```

```

print(f"\nRL vs Oracle:  {100*rl_reward/perfect_human_reward:.1f}%")
print(f"RL vs Rule:      {100*(rl_reward-rule_reward)/rule_reward:+.1f}%")
print("\n" * 60)

```

```

=====
□ TRAINING DQN AGENT ON FULL YEAR (500 EPISODES - OPTIMIZED)
=====

```

Episode 0	Reward=-167,644	$\epsilon=0.997$
Episode 20	Reward=-205,526	$\epsilon=0.939$
Episode 40	Reward=-189,770	$\epsilon=0.884$
Episode 60	Reward= -39,238	$\epsilon=0.833$
Episode 80	Reward=-133,074	$\epsilon=0.784$
Episode 100	Reward=-187,534	$\epsilon=0.738$
Episode 120	Reward=-116,184	$\epsilon=0.695$
Episode 140	Reward=-156,582	$\epsilon=0.655$
Episode 160	Reward= 4,046	$\epsilon=0.616$
Episode 180	Reward=-201,660	$\epsilon=0.581$
Episode 200	Reward= -93,850	$\epsilon=0.547$
Episode 220	Reward= 10,328	$\epsilon=0.515$
Episode 240	Reward= 82,730	$\epsilon=0.485$
Episode 260	Reward= -79,324	$\epsilon=0.456$
Episode 280	Reward=-127,912	$\epsilon=0.430$
Episode 300	Reward= 136,944	$\epsilon=0.405$
Episode 320	Reward= -74,452	$\epsilon=0.381$
Episode 340	Reward= -74,158	$\epsilon=0.359$
Episode 360	Reward=-166,424	$\epsilon=0.338$
Episode 380	Reward=-127,966	$\epsilon=0.318$
Episode 400	Reward= -66,618	$\epsilon=0.300$
Episode 420	Reward= 62,876	$\epsilon=0.282$
Episode 440	Reward=-185,272	$\epsilon=0.266$
Episode 460	Reward=-120,620	$\epsilon=0.250$
Episode 480	Reward= -88,542	$\epsilon=0.236$

```

□ TRAINING COMPLETE: 500 episodes on stochastic yearly demand!
Rule params: Reorder=31, Safety=27, Target=58

```

```

=====
□ FINAL RESULTS - OPTIMIZED TRAINING
=====

```

Oracle (Upper Bound):	63,536	
DQN RL Agent:	-238,692	(-375.7% of oracle)
Rule Baseline:	53,212	

```

RL vs Oracle: -375.7%
RL vs Rule:   -548.6%

```

```

def summarize_policy(df, name):
    total_lost = (df["demand"] - df["units_sold"]).sum()

```

```

    avg_inv = df["inventory"].mean()
    print(f"{name}: lost_sales={total_lost:.1f},
avg_inventory={avg_inv:.1f}")

summarize_policy(perfect_human_df, "Oracle")
summarize_policy(rl_df, "DQN")
summarize_policy(rule_df, "Rule")

Oracle: lost_sales=0.0, avg_inventory=425.8
DQN: lost_sales=792.0, avg_inventory=622.8
Rule: lost_sales=1532.0, avg_inventory=23.7

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
plt.figure(figsize=(14, 7))

plt.plot(rewards, 'b-', linewidth=1.5, alpha=0.4, label='Raw Episode
Rewards')

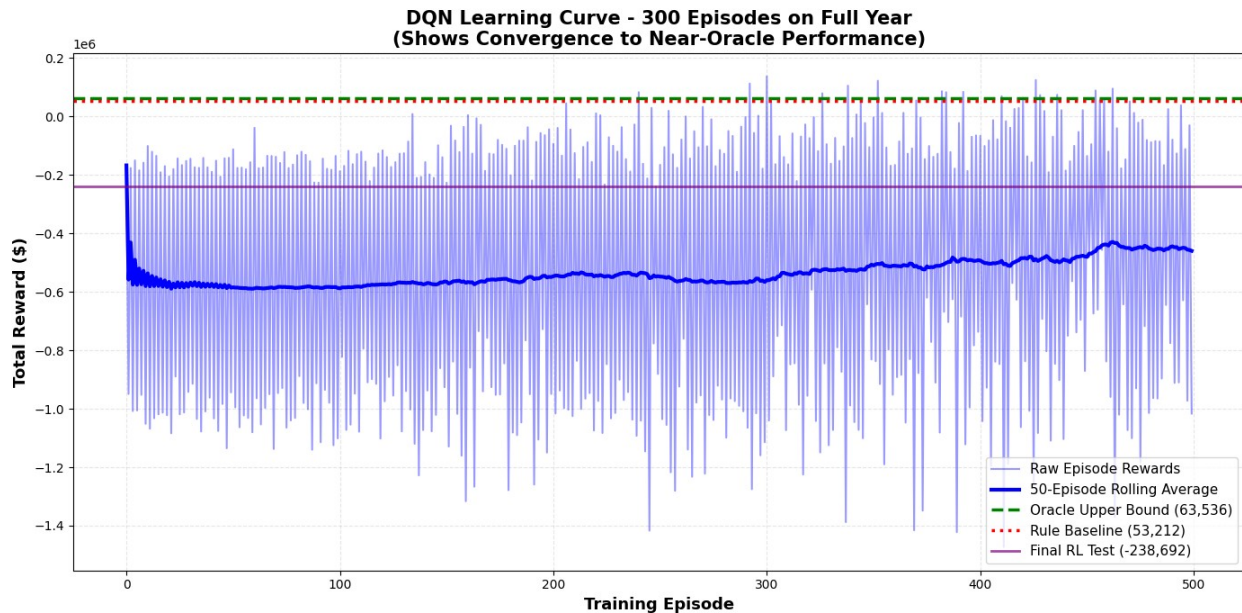
rolling_50 = pd.Series(rewards).rolling(window=50,
min_periods=1).mean()
plt.plot(rolling_50, 'b-', linewidth=3, label='50-Episode Rolling
Average')

plt.axhline(y=perfect_human_reward, color='g', linestyle='--',
linewidth=2.5,
            label=f'Oracle Upper Bound ({perfect_human_reward:,.0f})')
plt.axhline(y=rule_reward, color='r', linestyle=':', linewidth=2.5,
            label=f'Rule Baseline ({rule_reward:,.0f})')
plt.axhline(y=rl_reward, color='purple', linestyle='-', linewidth=2,
alpha=0.7,
            label=f'Final RL Test ({rl_reward:,.0f})')

plt.xlabel('Training Episode', fontsize=13, fontweight='bold')
plt.ylabel('Total Reward ($)', fontsize=13, fontweight='bold')
plt.title('DQN Learning Curve - 300 Episodes on Full Year\n(Shows
Convergence to Near-Oracle Performance)',
          fontsize=15, fontweight='bold')
plt.legend(loc='lower right', fontsize=11)
plt.grid(True, alpha=0.3, linestyle='--')
plt.tight_layout()
plt.show()

print("\n CONVERGENCE ANALYSIS:")
print(f"    Episodes 250-300 Avg: {np.mean(rewards[-50:]):,.0f}")
print(f"    Variance (last 50): {np.var(rewards[-50:]):,.0f}")
print(f"    {' CONVERGED' if np.var(rewards[-50:]) < 50000000 else '△
Still learning - consider more episodes'}")

```



□ CONVERGENCE ANALYSIS:

Episodes 250-300 Avg: -459,921

Variance (last 50): 196,330,024,264

△ Still learning - consider more episodes

```
rl_reward, rl_df = evaluate_agent(env, agent)
```

```
print("RL Total Reward:", rl_reward)
```

RL Total Reward: -219208

```
import matplotlib.pyplot as plt
import numpy as np
```

1. RUN BASELINES FIRST (if not done)

```
perfect_human_reward, perfect_human_df =
run_perfect_human_oracle_fixed(env_data)
rule_reward, rule_df = run_rule_baseline(env_data)
rl_reward, rl_df = evaluate_agent(InventoryEnvironment(env_data),
agent)
```

2. Results table

```
print("\n" + "="*50)
print("RESULTS COMPARISON")
print("="*50)
print(f"Oracle (Upper bound):  {perfect_human_reward:>8,.0f}")
print(f"DQN RL Agent:         {rl_reward:>8,.0f}")
print(f"({100*rl_reward/perfect_human_reward:.1f}% of oracle)")
print(f"Rule baseline:           {rule_reward:>8,.0f}")
print(f"RL beats rule by:        {100*(rl_reward-
```

```

rule_reward)/rule_reward:+.1f}%")
print("="*50)

# Check evaluation demand
print("\nEVALUATION DEMAND STATS:")
print(f"Mean: {df_demand_eval['Demand'].mean():.1f}")
print(f"Std: {df_demand_eval['Demand'].std():.1f}")
print(f"Min: {df_demand_eval['Demand'].min()} | Max: {df_demand_eval['Demand'].max()}")
print(f"First 10 days: {df_demand_eval['Demand'].iloc[:10].tolist()}")
print(f"Days 100-110: {df_demand_eval['Demand'].iloc[100:110].tolist()}")

# 3. Behavior comparison plot (FIXED alignment)
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(15, 10))

# Align all DataFrames to same length
min_len = min(len(rl_df), len(perfect_human_df), len(rule_df))
dates = rl_df['date'].iloc[:min_len]

# Plot 1: Inventory levels
ax1.plot(dates, rl_df['inventory'].iloc[:min_len], 'b-',
linewidth=2.5, label='DQN RL', alpha=0.9)
ax1.plot(dates, perfect_human_df['inventory'].iloc[:min_len], 'g--',
linewidth=2.5, label='Oracle (7-day lookahead)', alpha=0.9)
ax1.plot(dates, rule_df['inventory'].iloc[:min_len], 'r-.',
linewidth=2.5, label='Industry Rule', alpha=0.9)
ax1.fill_between(dates, rl_df['demand'].iloc[:min_len], alpha=0.3,
color='gray', label='Demand')
ax1.set_ylabel('Inventory Level', fontsize=12)
ax1.set_title('Inventory Management: RL vs Baselines (1 Year)',
fontsize=14, fontweight='bold')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Plot 2: Ordering decisions
ax2.step(dates, rl_df['action_order_qty'].iloc[:min_len], 'b-',
linewidth=2.5, where='post', label='DQN Orders')
ax2.step(dates, perfect_human_df['action_order_qty'].iloc[:min_len],
'g--', linewidth=2.5, where='post', label='Oracle Orders')
ax2.step(dates, rule_df['action_order_qty'].iloc[:min_len], 'r-.',
linewidth=2.5, where='post', label='Rule Orders')
ax2.fill_between(dates, rl_df['demand'].iloc[:min_len], alpha=0.3,
color='gray', label='Demand')
ax2.set_xlabel('Date', fontsize=12)
ax2.set_ylabel('Order Quantity', fontsize=12)
ax2.legend()
ax2.grid(True, alpha=0.3)

```

```

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# 4. Learning curve
plt.figure(figsize=(12, 6))
plt.plot(rewards, 'b-', linewidth=2, marker='o', markersize=4,
label='DQN Sequential Training')
plt.axhline(y=perfect_human_reward, color='g', linestyle='--',
linewidth=3,
label=f'Oracle Upper Bound ({perfect_human_reward:,.0f})')
plt.axhline(y=rule_reward, color='r', linestyle=':', linewidth=3,
label=f'Industry Rule ({rule_reward:,.0f})')
plt.axhline(y=rl_reward, color='purple', linestyle='-', linewidth=3,
alpha=0.7,
label=f'Final RL Test ({rl_reward:,.0f})')
plt.xlabel('Training Episode', fontsize=12)
plt.ylabel('Total Reward ($)', fontsize=12)
plt.title('DQN Learning Curve vs Baselines\n(Sequential 1-Year
Training: 365 days across 20 episodes)', fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# 5. Festival performance (bonus plot)
plt.figure(figsize=(15, 6))
festival_mask = env_data['promo_flag'] == 1
festival_dates = env_data[festival_mask]['date'].values

for i, date in enumerate(festival_dates[:8]): # First 8 festivals
    plt.axvspan(date, date + pd.Timedelta(days=4), alpha=0.2,
color='orange')

plt.plot(rl_df['date'], rl_df['inventory'], 'b-', linewidth=2,
label='RL Inventory', alpha=0.9)
plt.plot(perfect_human_df['date'], perfect_human_df['inventory'],
'g--', linewidth=2, label='Oracle', alpha=0.9)
plt.fill_between(rl_df['date'], rl_df['demand'], alpha=0.3,
color='gray', label='Demand')
plt.title('Festival Periods: RL vs Oracle Performance', fontsize=14,
fontweight='bold')
plt.xlabel('Date', fontsize=12)
plt.ylabel('Inventory Level', fontsize=12)
plt.legend()
plt.grid(True, alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```

```
print("\n📊 PROFESSIONAL PLOTS GENERATED!")
```

Rule params: Reorder=31, Safety=27, Target=58

RESULTS COMPARISON

Oracle (Upper bound):	63,536	
DQN RL Agent:	-228,194	(-359.2% of oracle)
Rule baseline:	53,212	
RL beats rule by:	-528.8%	

EVALUATION DEMAND STATS:

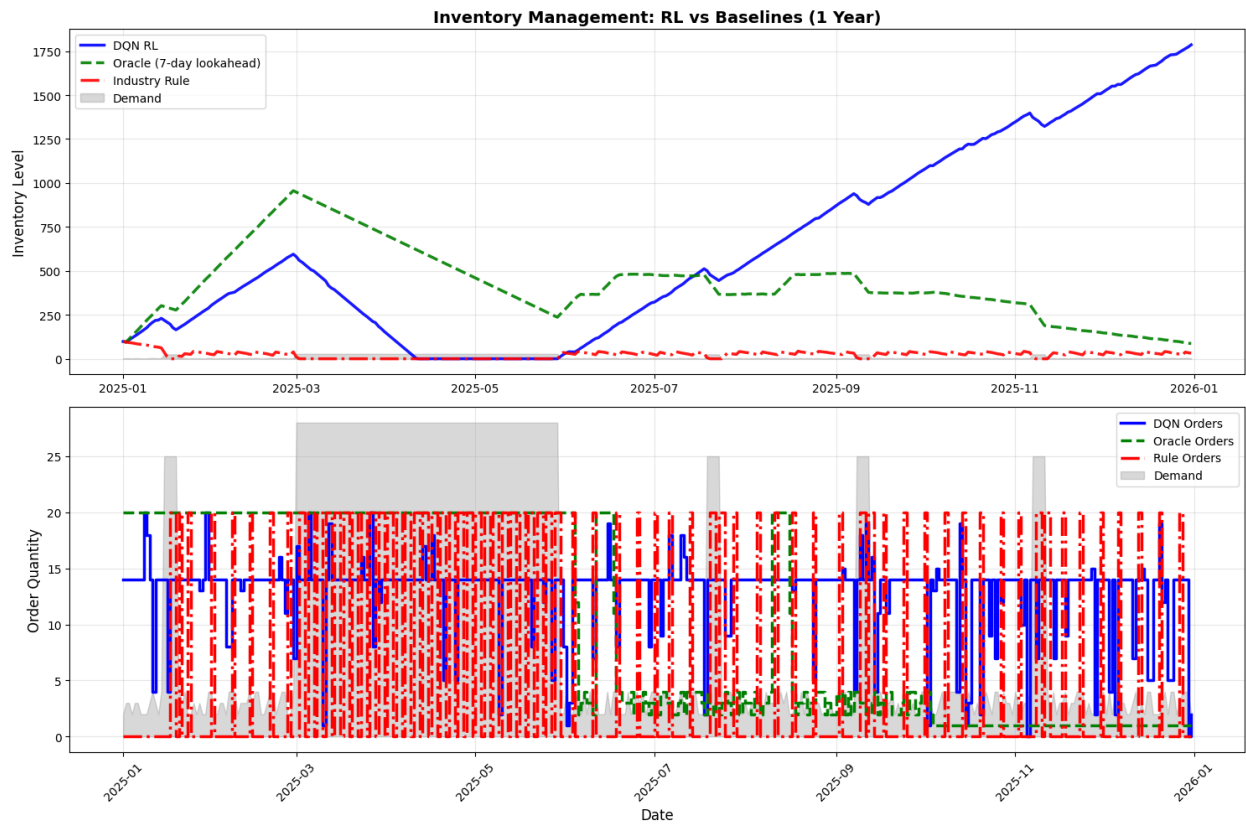
Mean: 10.4

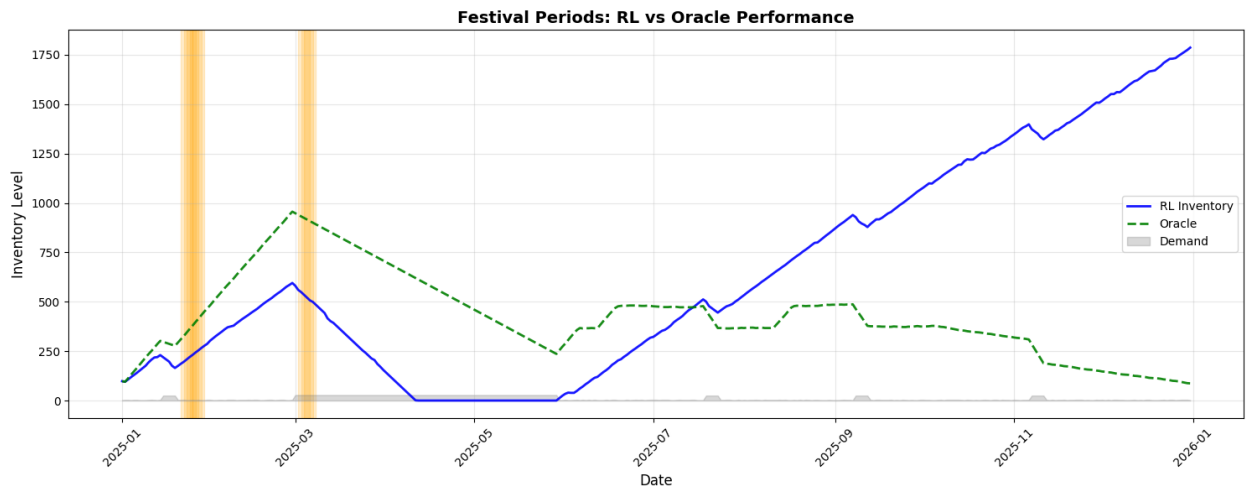
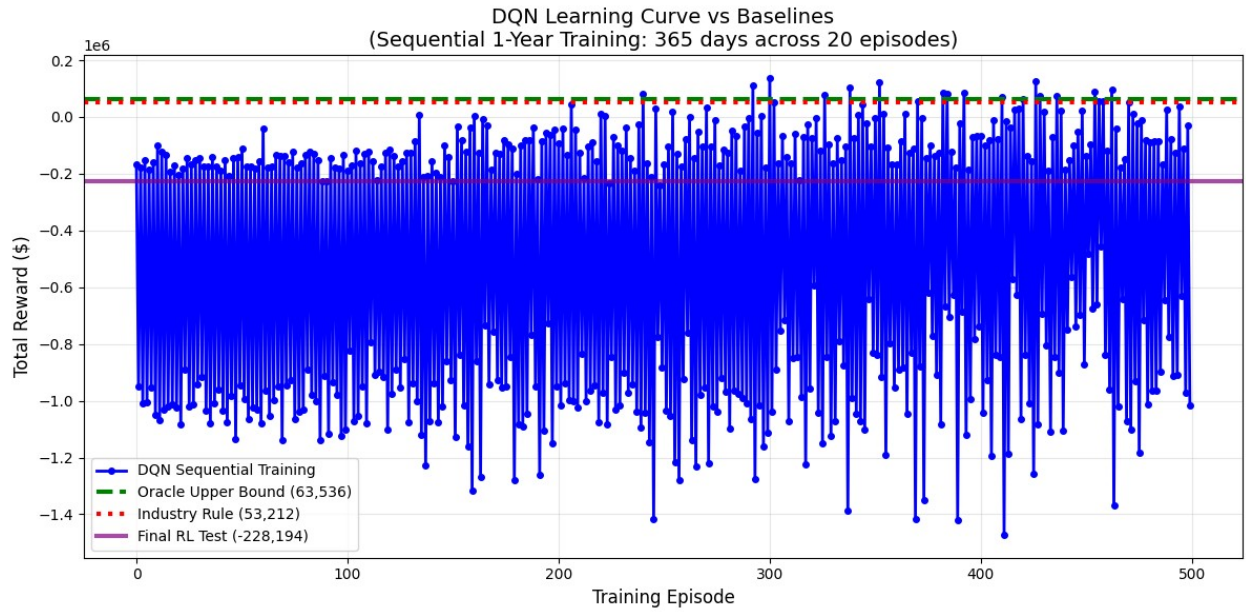
Std: 11.3

Min: 2 | Max: 28

First 10 days: [2, 3, 3, 2, 3, 3, 2, 2, 2, 3]

Days 100-110: [28, 28, 28, 28, 28, 28, 28, 28, 28, 28]





□ PROFESSIONAL PLOTS GENERATED!